第17章 调节磁盘和CPU的矛盾——InnoDB的Buffer Pool

17.1 缓存的重要性

通过前面章节的唠叨我们知道，对于使用 InnoDB 存储引擎的表来说，无论是用于存储用户数据的索引（包括聚簇索引和二级索引），还是各种系统数据，都是以页的形式存放在表空间中。所谓的表空间，只不过是 InnoDB 对一个或几个实际文件的抽象。也就是说，我们的数据说到底还是存储在磁盘上。各位同学也都知道，磁盘的速度慢得“跟乌龟一样”，怎么能配得上“快如风，疾如电”的 CPU 呢？所以 InnoDB 存储引擎在处理客户端的请求时，如果需要访问某个页的数据，就会把完整的页中的数据全部加载到内存中。也就是说，即使只需要访问一个页的一条记录，也需要先把整个页的数据加载到内存中。将整个页加载到内存中后就可以进行读写访问了，而且在读写访问之后并不着急把该页对应的内存空间释放掉，而是将其缓存起来，这样将来有请求再次访问该页面时，就可以省下磁盘 I/O 的开销了。

17.2 InnoDB 的 Buffer Pool

17.2.1 啥是 Buffer Pool

为了缓存磁盘中的页，设计 InnoDB 的大叔在 MySQL 服务器启动时就向操作系统申请了一片连续的内存，他们给这片内存起了个名字——Buffer Pool（缓冲池）。它有多大呢？这个其实取决于我们机器的配置：如果你是“土豪”，有 512GB 内存，分配个几百 GB 作为 Buffer Pool 当然没有问题；如果没那么有钱，设置得小一点也问题不大。默认情况下，Buffer Pool 只有 128MB。如果嫌弃这个 128MB 太大或者太小，可以在启动服务器的时候配置 innodb_buffer_pool_size 启动选项（这个启动选项表示 Buffer Pool 的大小）的值，就像下面这样：

```
[server]
innodb_buffer_pool_size = 268435456
```

innodb_buffer_pool_size 的单位是字节，所以上面的配置指定了 Buffer Pool 的大小为 256MB。需要注意的是，Buffer Pool 也不能太小，最小值为 5MB（当 innodb_buffer_pool_size 的值小于 5M 时会自动设置成 5MB）。

17.2.2 Buffer Pool 内部组成

Buffer Pool 对应的一片连续的内存被划分为若干个页面，页面大小与 InnoDB 表空间使

用的页面大小一致，默认都是 16KB。为了与磁盘中的页面区分开来，我们这里把这些 Buffer Pool 中的页面称为缓冲页。为了更好地管理 Buffer Pool 中的这些缓冲页，设计 InnoDB 的大叔为每一个缓冲页都创建了一些控制信息。这些控制信息包括该页所属的表空间编号、页号、缓冲页在 Buffer Pool 中的地址、链表节点信息等。除了这些信息之外，当然还有一些别的控制信息，我们这就不全唠叨一遍了，等用到了再说。

每个缓冲页对应的控制信息占用的内存大小是相同的，我们把每个页对应的控制信息占用的一块内存称为一个控制块。控制块与缓冲页是一一对应的，它们都存放到 Buffer Pool 中。其中控制块存放到 Buffer Pool 的前面，缓冲页存放到 Buffer Pool 的后面，所以整个 Buffer Pool 对应的内存空间看起来如图 17-1 所示。

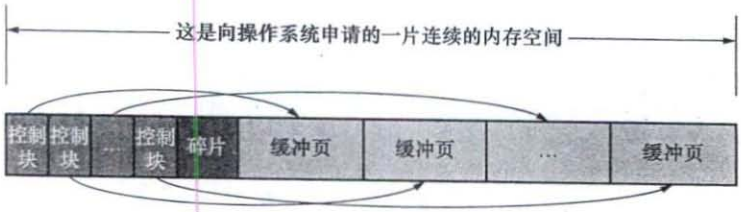

图 17-1 Buffer Pool 对应的内存空间

咦？控制块和缓冲页之间的那个碎片是什么玩意儿？大家想想看，每一个控制块都对应一个缓冲页，那么在分配足够多的控制块和缓冲页后，剩余的那点儿空间可能不够一对控制块和缓冲页的大小，自然也就用不到了。这个用不到的内存空间就称为碎片。当然，如果把 Buffer Pool 的大小设置得刚刚好，也可能不会产生碎片。

小贴士

在 DEBUG 模式下，每个控制块大约占用缓冲页大小的 5%（非 DEBUG 模式下会更小一点）。在 MySQL 5.7.22 版本的 DEBUG 模式下，每个控制块占用的大小是 808 字节。而我们设置的 innodb_buffer_pool_size 并不包含这部分控制块占用的内存空间大小，也就是说 InnoDB 在为 Buffer Pool 向操作系统申请连续的内存空间时，这片连续的内存空间会比 innodb_buffer_pool_size 的值大 5% 左右。

17.2.3 free 链表的管理

当我们最初启动 MySQL 服务器的时候，需要完成 Buffer Pool 的初始化过程。就是先向操作系统申请 Buffer Pool 的内存空间，然后把它划分成若干对控制块和缓冲页。但是此时并没有真实的磁盘页被缓存到 Buffer Pool 中（因为还没有用到），之后随着程序的运行，会不断地有磁盘上的页被缓存到 Buffer Pool 中。

那么问题来了，从磁盘上读取一个页到 Buffer Pool 中时，该放到哪个缓冲页的位置呢？或者说怎么区分 Buffer Pool 中哪些缓冲页是空闲的，哪些已经被使用了呢？我们最好在某个地方记录 Buffer Pool 中哪些缓冲页是可用的。这个时候缓冲页对应的控制块就派上大用场了——我们可以把所有空闲的缓冲页对应的控制块作为一个节点放到一个链表中，这个链表也可以称为 free 链表（或者说空闲链表）。刚刚完成初始化的 Buffer Pool 中，所有的缓冲页都是空闲的，所以每一个缓冲页对应的控制块都会加入到 free 链表中。假设该 Buffer Pool 中可容

纳的缓冲页数量为 n，那么增加了 free 链表的效果图如图 17-2 所示。

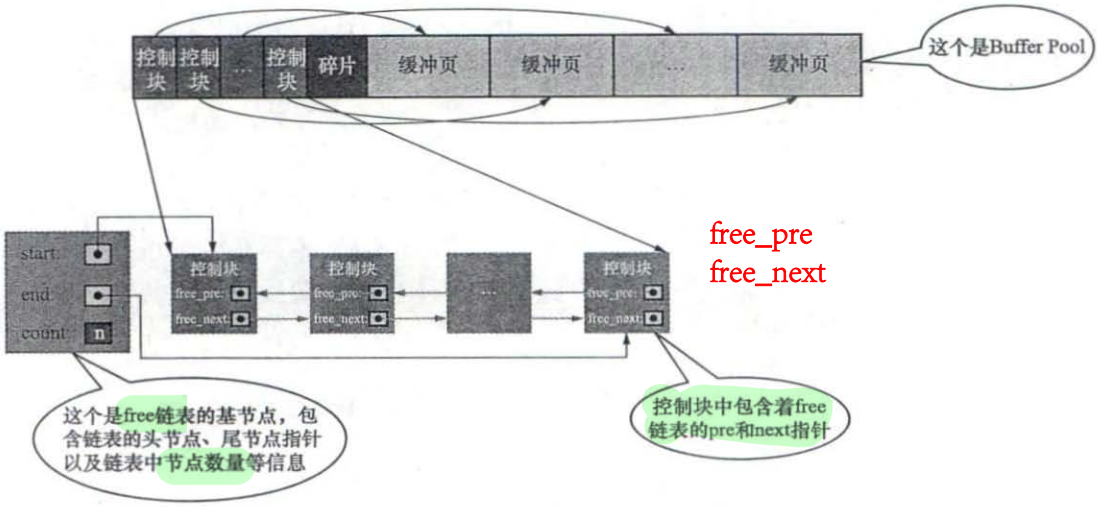

图 17-2　free 链表的效果图

从图 17-2 可以看出，为了管理好这个 free 链表，我们特意为这个链表定义了一个基节点，里面包含链表的头节点地址、尾节点地址，以及当前链表中节点的数量等信息。这里需要注意的是，链表的基节点占用的内存空间并不包含在为 Buffer Pool 申请的一大片连续内存空间之内，而是一块单独申请的内存空间。

小贴士

链表基节点占用的内存空间并不大，在 MySQL 5.7.22 版本中，每个基节点只占用 40 字节。后面即将介绍的许多不同的链表中，它们的基节点的内存分配方式与 free 链表的基节点是一样的，都是一块单独申请的 40 字节的内存空间，并不包含在为 Buffer Pool 申请的一大片连续内存空间之内。

有了这个 free 链表之后事儿就好办了，每当需要从磁盘中加载一个页到 Buffer Pool 中时，就从 free 链表中取一个空闲的缓冲页，并且把该缓冲页对应的控制块的信息填上（就是该页所在的表空间、页号之类的信息），然后把该缓冲页对应的 free 链表节点（也就是对应的控制块）从链表中移除，表示该缓冲页已经被使用了。

小贴士

“从链表中取一个缓冲页对应的控制块”这样的陈述有点儿繁琐，在后边某些场景下我们可能会将其简称为“从链表中取一个缓冲页”，大家心里要清楚我们真正从链表中获取的是控制块，通过控制块可以访问到真正的页就好了。同理，“遍历 Buffer Pool 中的缓冲页”的意思其实是“遍历 Buffer Pool 中各个缓冲页对应的控制块”。

缓冲块是顺序紧挨着的，所以可以直接遍历

17.2.4　缓冲页的哈希处理

前文说过，当我们需要访问某个页中的数据时，就会把该页从磁盘加载到 Buffer Pool 中。如果该页已经在 Buffer Pool 中的话，直接使用就可以了。那么问题也就来了，我们怎么知道该页在不在 Buffer Pool 中呢？难不成需要依次遍历 Buffer Pool 中的各个缓冲页么？一个 Buffer Pool 中的缓冲页这么多，都遍历完岂不是要累死？

再回头想想，我们其实是根据表空间号 + 页号来定位一个页的，也就相当于表空间号 + 页号是一个 key（键），缓冲页控制块就是对应的 value（值）。怎么通过一个 key 来快速找到一个 value 呢？当然是哈希表了！

小贴士

啥？别告诉我你不知道哈希表是啥？咱们这一章甚至这本书都不讲哈希表的知识。如果你不知道，就去找本数据结构的书看看吧。

所以我们可以用表空间号 + 页号作为 key，用缓冲页控制块的地址作为 value 来创建一个哈希表。在需要访问某个页的数据时，先从哈希表中根据表空间号 + 页号看看是否有对应的缓冲页。如果有，直接使用该缓冲页就好；如果没有，就从 free 链表中选一个空闲的缓冲页，然后把磁盘中对应的页加载到该缓冲页的位置。

17.2.5 flush 链表的管理

如果我们修改了 Buffer Pool 中某个缓冲页的数据，它就与磁盘上的页不一致了，这样的缓冲页也称为脏页（dirty page）。当然，我们可以每当修改完某个缓冲页时，就立即将其刷新到磁盘中对应的页上。但是频繁地往磁盘中写数据会严重影响程序的性能（毕竟磁盘慢得“像乌龟一样”）。所以每次修改缓冲页后，我们并不着急立即把修改刷新到磁盘上，而是在未来的某个时间点进行刷新。至于这个刷新的时间点会在后面进行说明，现在先不用管。

但是，如果不立即将修改刷新到磁盘，那之后再刷新的时候我们怎么知道 Buffer Pool 中哪些页是脏页，哪些页从来没被修改过呢？总不能把所有的缓冲页都刷新到磁盘上吧。假如 Buffer Pool 被设置得很大，比如有 300GB，那么一次性刷新这么多数据岂不是要慢死！所以，我们不得不再创建一个存储脏页的链表，凡是被修改过的缓冲页对应的控制块都会作为一个节点加入到这个链表中。因为这个链表节点对应的缓冲页都是需要被刷新到磁盘上的，所以也称为 flush 链表。flush 链表的构造与 free 链表差不多。假设 Buffer Pool 在某个时间点的脏页数量为 n，那么对应的 flush 链表如图 17-3 所示。

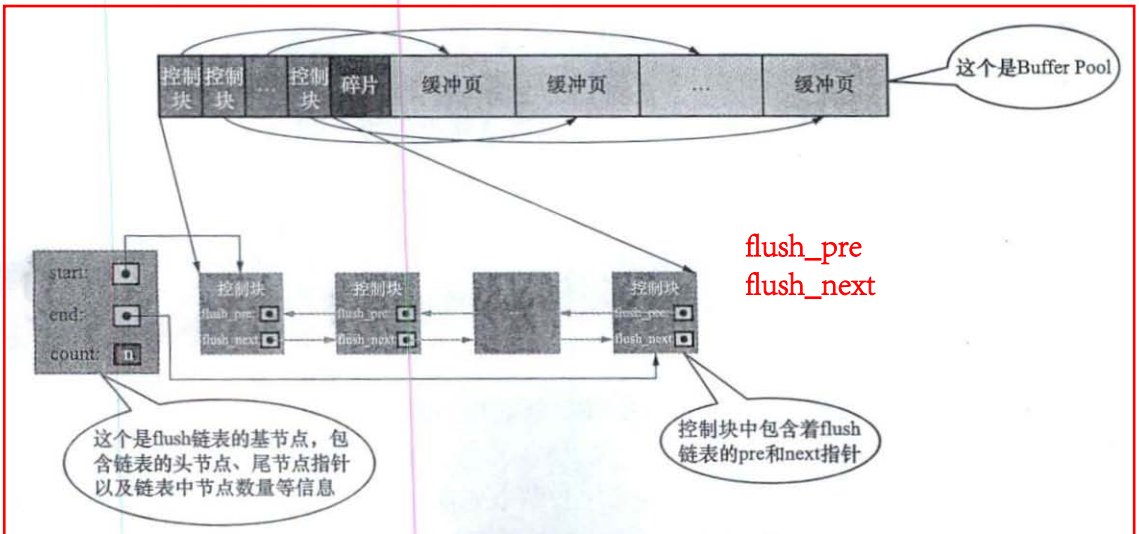

图 17-3 Buffer Pool 中的脏页数量为 n 时对应的 flush 链表

小贴士

如果一个缓冲页是空闲的，那它肯定不可能是脏页。如果一个缓冲页是脏页，那它肯定就不是空闲的。也就是说，某个缓冲页对应的控制块不可能既是free链表的节点，也是flush链表的节点。

17.2.6 LRU链表的管理

1. 缓冲区不够的窘境

Buffer Pool对应的内存大小毕竟是有限的。如果需要缓存的页占用的内存大小超过了Buffer Pool的大小，也就是free链表中已经没有多余的空闲缓冲页了，这岂不是很尴尬！发生了这样的事儿该咋办？当然是把某些旧的缓冲页从Buffer Pool中移除，然后再把新的页放进来。那么问题来了：移除哪些缓冲页呢？

为了回答这个问题，我们还需要回到设立Buffer Pool的初衷——想减少磁盘I/O，最好每次在访问某个页的时候它已经被加载到Buffer Pool中了。假设我们一共访问了n次页，那么被访问的页已经在Buffer Pool中的次数除以n就是Buffer Pool命中率。我们的期望是Buffer Pool命中率越高越好。从这个角度出发，回想一下我们的微信聊天列表，排在前面的都是最近频繁使用的，排在后面的自然就是最近很少使用的。假如列表能容纳的联系人有限，你是把最近很频繁使用的留下，还是把最近很少使用的留下呢？当然是留下最近很频繁使用的了。

2. 简单的LRU链表

管理Buffer Pool的缓冲页其实也是这个道理。当Buffer Pool中不再有空闲的缓冲页时，就需要淘汰掉最近很少使用的部分缓冲页。不过，我们怎么知道哪些缓冲页最近频繁使用，哪些最近很少使用呢？神奇的链表再一次派上了用场。我们可以再创建一个链表，由于这个链表是为了按照最近最少使用的原则去淘汰缓冲页的，所以这个链表可以被称为LRU（Least Recently Used）链表。当需要访问某个页时，可以按照下面的方式处理LRU链表：

- 如果该页不在Buffer Pool中，在把该页从磁盘加载到Buffer Pool中的缓冲页时，就把该缓冲页对应的控制块作为节点塞到LRU链表的头部；
- 如果该页已经被加载到Buffer Pool中，则直接把该页对应的控制块移动到LRU链表的头部。

也就是说，只要我们使用到某个缓冲页，就把该缓冲页调整到LRU链表的头部，这样LRU链表尾部就是最近最少使用的缓冲页了。所以，当Buffer Pool中的空闲缓冲页使用完时，到LRU链表的尾部找些缓冲页淘汰掉就OK了。真简单！

3. 划分区域的LRU链表

我们高兴的太早了。上面这个简单的LRU链表用了没多长时间就发现问题了。它存在下面这两种比较尴尬的情况。

- 情况1：InnoDB提供了一个看起来比较贴心的服务——预读（read ahead）。我们前边说过只有当我们用到某个页时，才会将其从磁盘加载到Buffer Pool中，用不到则不加

载。所谓预读，就是 InnoDB 认为执行当前的请求时，可能会在后面读取某些页面，于是就预先把这些页面加载到 Buffer Pool 中。根据触发方式的不同，预读又可以细分为下面两种。

- 线性预读：设计 InnoDB 的大叔提供了一个系统变量 innodb_read_ahead_threshold，如果顺序访问的某个区（extent）的页面超过这个系统变量的值，就会触发一次异步读取下一个区中全部的页面到 Buffer Pool 中的请求。注意异步读取意味着从磁盘中加载这些被预读的页面时，并不会影响到当前工作线程的正常执行。innodb_read_ahead_threshold 系统变量的值默认是 56，我们可以在服务器启动时通过启动选项来调整该值，或者在服务器运行过程中直接调整该系统变量的值。由于它是一个全局变量，因此要使用 SET GLOBAL 命令来修改。
- 随机预读：如果某个区的 13 个连续的页面都被加载到了 Buffer Pool 中，无论这些页面是不是顺序读取的，都会触发一次异步读取本区中所有其他页面到 Buffer Pool 中的请求。设计 InnoDB 的大叔同时提供了 innodb_random_read_ahead 系统变量，它的默认值为 OFF，也就意味着 InnoDB 并不会默认开启随机预读的功能。如果想开启该功能，可以通过修改启动选项或者直接使用 SET GLOBAL 命令把该变量的值设置为 ON。 这13个页面还需要是young中的1/4前面的

预读本来是个好事儿，如果预读到 Buffer Pool 中的页被成功地使用到，那就可以极大地提高语句执行的效率。可是如果用不到呢？这些预读的页都会放到 LRU 链表的头部。但是，如果此时 Buffer Pool 的容量不太大，而且很多预读的页面都没有用到的话，就会导致处于 LRU 链表尾部的一些缓冲页会很快被淘汰掉，从而大大降低 Buffer Pool 命中率。

- 情况 2：有的小伙伴可能会写一些需要进行全表扫描的语句（比如在没有建立合适的索引或者压根儿没有 WHERE 子句的查询时）。

全表扫描意味着什么？意味着将访问该表的聚簇索引的所有叶子节点对应的页（当然，扫描叶子节点时，首先需要从 B+ 树中定位到第一个叶子节点的第一条记录。这个过程还得访问一些内节点）！如果需要访问的页面特别多，而 Buffer Pool 又不能全部容纳它们的话，这就意味着需要将其他语句在执行过程中用到的页面“排挤”出 Buffer Pool，之后在其他语句重新执行时，又需要重新将需要用到的页从磁盘加载到 Buffer Pool 中（这就像我在一个饭店吃着好好的，忽然来了一群人把我从饭店中赶了出去，等他们吃完之后我又得重新点菜吃）。

我们在业务中一般不对很大的表执行全表扫描操作，这是一个很耗时的操作，只有在特定场景下偶尔对很大的表执行全表扫描操作。由于对很大的表执行全表扫描操作可能要把 Buffer Pool 中的缓冲页换一次，这会严重影响到其他查询对 Buffer Pool 的使用，从而降低了 Buffer Pool 命中率。

一言蔽之，可能降低 Buffer Pool 命中率的两种情况如下所示：

- 加载到 Buffer Pool 中的页不一定被用到；
- 如果有非常多的使用频率偏低的页被同时加载到 Buffer Pool 中，则可能会把那些使用频率非常高的页从 Buffer Pool 中淘汰掉。

因为这两种情况的存在，设计 InnoDB 的大叔把这个 LRU 链表按照一定比例分成两截：

- 一部分存储使用频率非常高的缓冲页；这一部分链表也称为热数据，或者称为 young 区域；
- 另一部分存储使用频率不是很高的缓冲页；这一部分链表也称为冷数据，或者称为 old 区域。

为了方便大家理解，我们把示意图进行了简化，如图 17-4 所示。

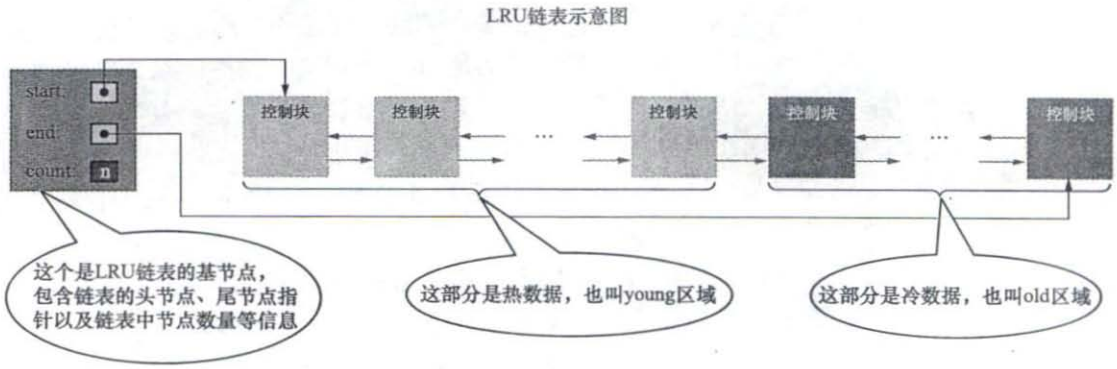

图 17-4　LRU 链表示意图

需要特别注意的一点是，我们是按照某个比例将 LRU 链表分成两半的，而不是某些节点固定位于 young 区域，某些节点固定位于 old 区域。随着程序的运行，某个节点所属的区域也可能发生变化。那么，这个划分成两截的比例是怎么确定的呢？对于 InnoDB 存储引擎来说，我们可以通过查看系统变量 innodb_old_blocks_pct 的值来确定 old 区域在 LRU 链表中所占的比例。比如下面这样：

```
mysql> SHOW VARIABLES LIKE 'innodb_old_blocks_pct';
+-----------------------+-------+
| Variable_name         | Value |
+-----------------------+-------+
| innodb_old_blocks_pct | 37    |
+-----------------------+-------+
1 row in set (0.01 sec)
```

从结果可以看出，默认情况下 old 区域在 LRU 链表中所占的比例是 37%。也就是说，old 区域大约占 LRU 链表的 3/8。这个比例是可以进行设置的，我们可以在启动服务器时通过修改 innodb_old_blocks_pct 启动选项来控制 old 区域在 LRU 链表中所占的比例。比如在配置文件中书写下面的语句：

```
[server]
innodb_old_blocks_pct = 40
```

这样在启动服务器后，old 区域占 LRU 链表的比例就是 40%。当然，在服务器运行期间也可以修改这个系统变量的值。不过需要注意的是，这个系统变量属于全局变量，所以我们需要使用 SET GLOBAL 命令来修改：

```
SET GLOBAL innodb_old_blocks_pct = 40;
```

有了这个被划分成 young 和 old 区域的 LRU 链表之后，设计 InnoDB 的大叔就可以针对前文提到的两种可能降低 Buffer Pool 命中率的情况进行优化了。

- 针对预读的页面可能不进行后续访问的优化。

设计 InnoDB 的大叔规定，当磁盘上的某个页面在初次加载到 Buffer Pool 中的某个缓冲页时，该缓冲页对应的控制块会放到 old 区域的头部。这样一来，预读到 Buffer Pool 却不进行后续访问的页面就会被逐渐从 old 区域逐出，而不会影响 young 区域中使用比较频繁的缓冲页。

- 针对全表扫描时，短时间内访问大量使用频率非常低的页面的优化。

在进行全表扫描时，虽然首次加载到 Buffer Pool 中的页放到了 old 区域的头部，但是后续会被马上访问到，每次进行访问时又会把该页放到 young 区域的头部，这样仍然会把那些使用频率比较高的页面给“排挤”下去。有的读者会想：是否可以在第一次访问该页面时不将其从 old 区域移动到 young 区域的头部，而是在后续访问时再将其移动到 young 区域的头部？回答是：行不通！因为设计 InnoDB 的大叔规定，每次去页面中读取一条记录时，都算是访问一次页面。而一个页面中可能会包含很多条记录，也就是说读取完某个页面的记录就相当于访问了这个页面好多次。

咋办？全表扫描有一个特点，那就是它的执行频率非常低，谁也不会没事儿写全表扫描的语句玩儿。而且在执行全表扫描的过程中，即使某个页面中有很多条记录，尽管每读取一条记录都算是访问一次页面，但是这个过程所花费的时间也是非常少的。所以我们只需要规定，在对某个处于 old 区域的缓冲页进行第一次访问时，就在它对应的控制块中记录下这个访问时间，如果后续的访问时间与第一次访问的时间在某个时间间隔内，那么该页面就不会从 old 区域移动到 young 区域的头部，否则将它移动到 young 区域的头部。这个间隔时间是由系统变量 innodb_old_blocks_time 控制的，我们可以看一下：

```
mysql> SHOW VARIABLES LIKE 'innodb_old_blocks_time';
+-------------------------+-------+
| Variable_name           | Value |
+-------------------------+-------+
| innodb_old_blocks_time | 1000  |
+-------------------------+-------+
1 row in set (0.01 sec)
```

这个 innodb_old_blocks_time 变量的默认值是 1,000，单位是 ms，也就意味着对于从磁盘加载到 LRU 链表中 old 区域的某个页来说，如果第一次和最后一次访问该页面的时间间隔小于 1s，那么该页是不会加入到 young 区域的。很明显，在一次全表扫描的过程中，多次访问一个页面（也就是读取同一个页面中的多条记录）的时间不会超过 1s。当然，与 innodb_old_blocks_pct 一样，我们也可以在服务器启动或运行时设置 innodb_old_blocks_time 的值。这里就不赘述了，大家自己试试吧。这里需要注意的是，如果把 innodb_old_blocks_time 的值设置为 0，那么每次访问一个页面时，就会把该页面放到 young 区域的头部。

综上所述，正是因为将 LRU 链表划分为 young 区域和 old 区域这两个部分，又添加了 innodb_old_blocks_time 系统变量，预读机制和全表扫描造成的 Buffer Pool 命中率降低的问题才得到了遏制——因为用不到的预读页面以及全表扫描的页面都只会放到 old 区域，而不影响 young 区域中的缓冲页。

4. 更进一步优化 LRU 链表

LRU 链表这就说完了么？没有，早着呢！对于 young 区域的缓冲页来说，我们每次访问一个缓冲页就要把它移动到 LRU 链表的头部，这样开销是不是太大了。毕竟在 young 区域的缓冲页都是热点数据，也就是可能会经常访问。这样频繁地对 LRU 链表执行节点移动操作是不是不太好啊？是的，为了解决这个问题，其实我们还可以提出一些优化策略，比如只有被访问的缓冲页位于 young 区域 1/4 的后面时，才会被移动到 LRU 链表头部。这样就可以降低调整 LRU 链表的频率，从而提升性能（也就是说，如果某个缓冲页对应的节点在 young 区域的 1/4 中，再次访问该缓冲页时也不会将其移动到 LRU 链表头部）。

小贴士

前文介绍随机预读时曾提到，如果 Buffer Pool 中有某个区的 13 个连续页面就会触发随机预读。其实这是不严谨的，其实还要求这 13 个页面是非常热的页面。所谓的“非常热”，指的是这些页面在整个 young 区域的头 1/4 处。

还有没有针对 LRU 链表的其他优化措施呢？当然有啊，相关内容足够咱们写篇论文甚至写本书了。受限于篇幅以及考虑到大家的阅读体验，这里就适可而止了。大家如果想了解更多的优化知识，可以自己去阅读源码或者更多关于 LRU 链表的知识了。但是无论怎么优化，千万别忘了我们的初心：尽量高效地提高 Buffer Pool 命中率。

小贴士

只要从磁盘中加载一个页面到 Buffer Pool 的一个缓冲页中，该缓冲页对应的控制块就会作为一个节点加入到 LRU 链表中，这样一来，该缓冲页对应的控制块也就不在 free 链表中了。不过 flush 链表中的节点（控制块）肯定也是 LRU 链表中的节点。

17.2.7 其他的一些链表

为了更好地管理 Buffer Pool 中的缓冲页，除了前面提到的这些措施，设计 InnoDB 的大叔们还引进了其他一些链表。比如用于管理解压页的 unzip LRU 链表，用于管理压缩页的 zip clean 链表，zip free 数组中每一个元素都代表一个链表，它们组成伙伴系统来为压缩页提供内存空间等。反正是为了更好地管理这个 Buffer Pool 而引入了各种链表或其他数据结构，具体的使用方式我们也不啰嗦了。大家如果有兴趣深究，可以再去找一些更深的图书或者直接阅读源码。

小贴士

我们压根儿也没有深入唠叨过 InnoDB 中的压缩页，上面这些链表也只是为了内容的完整性而顺便提一下。如果大家看不懂也千万不要郁闷，因为我压根儿就没打算介绍它们。

17.2.8 刷新脏页到磁盘

后台有专门的线程负责每隔一段时间就把脏页刷新到磁盘，这样可以不影响用户线程处理正常的请求。刷新方式主要有下面两种。

- 从 LRU 链表的冷数据中刷新一部分页面到磁盘。

后台线程会定时从 LRU 链表尾部开始扫描一些页面，扫描的页面数量可以通过系统变量 innodb_lru_scan_depth 来指定。如果在 LRU 链表中发现脏页，则把它们刷新到磁盘。这种刷新页面的方式称为 BUF_FLUSH_LRU。

小贴士

一个缓冲页对应的控制块占用了很大的存储空间，其中就会存储诸如该缓冲页是否被修改的信息，所以在扫描 LRU 链表时，可以很轻松地获取到某个缓冲页是否是脏页的信息。

- 从 flush 链表中刷新一部分页面到磁盘。

后台线程也会定时从 flush 链表中刷新一部分页面到磁盘，刷新的速率取决于当时系统是否繁忙。这种刷新页面的方式称为 BUF_FLUSH_LIST。

小贴士

为了更高效地执行脏页刷盘操作，设计 InnoDB 的大叔还设计了许多系统变量来控制刷新的过程，比如 innodb_flush_neighbors、innodb_io_capacity_max、innodb_adaptive_flushing、innodb_max_dirty_pages_pct 等。至于这些系统变量是如何控制刷盘行为的，这并不是本书的内容，大家可以查阅官方文档。

有时，后台线程刷新脏页的进度比较慢，导致用户线程在准备加载一个磁盘页到 Buffer Pool 中时没有可用的缓冲页。这时就会尝试查看 LRU 链表尾部，看是否存在可以直接释放掉的未修改缓冲页。如果没有，则不得不将 LRU 链表尾部的一个脏页同步刷新到磁盘（与磁盘交互是很慢的，这会降低处理用户请求的速度）。这种将单个页面刷新到磁盘中的刷新方式称为 BUF_FLUSH_SINGLE_PAGE。

当然，在系统特别繁忙时，也可能出现用户线程从 flush 链表中刷新脏页的情况。很显然，在处理用户请求的过程中去刷新脏页是一种严重降低处理速度的行为（毕竟磁盘的速度太慢了）。这属于一种迫不得已的情况，后文在唠叨 redo 日志的 checkpoint 时，再进一步解释这一点。

17.2.9 多个 Buffer Pool 实例

前文说过，Buffer Pool 的本质是 InnoDB 向操作系统申请的一块连续的内存空间。在多线程环境下，访问 Buffer Pool 中的各种链表都需要加锁处理。在 Buffer Pool 特别大并且多线程并发访问量特别高的情况下，单一的 Buffer Pool 可能会影响请求的处理速度。所以在 Buffer Pool 特别大时，可以把它们拆分成若干个小的 Buffer Pool，每个 Buffer Pool 都称为一个实例。它们都是独立的——独立地申请内存空间、独立地管理各种链表……在多线程并发访问时并不会相互影响，从而提高了并发处理能力。我们可以在服务器启动的时候通过设置 innodb_buffer_pool_instances 的值来修改 Buffer Pool 实例的个数。比如下面这样：

```
[server]
innodb_buffer_pool_instances = 2
```

这表明我们要创建 2 个 Buffer Pool 实例，示意图如图 17-5 所示。

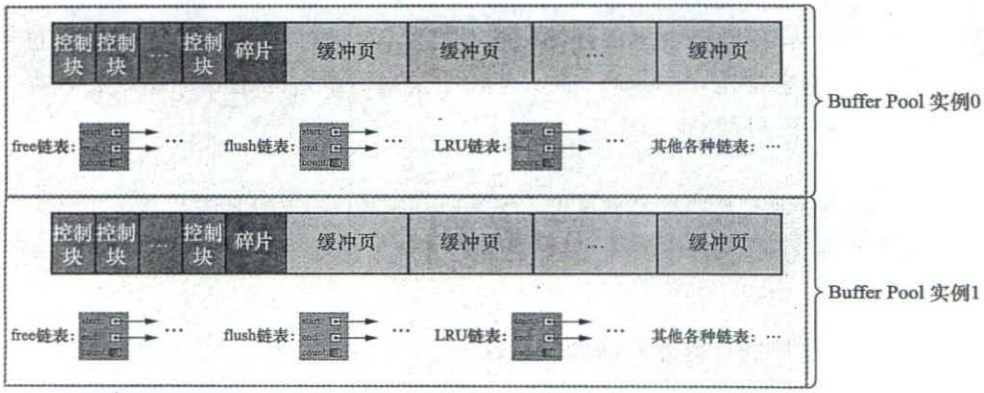

图 17-5　Buffer Pool 实例示意图

小贴士

简便起见，这里只画出了各个链表的基节点，大家应该清楚这些链表的节点其实就是每个缓冲页对应的控制块！

那么，每个 Buffer Pool 实例实际占多少内存空间呢？其实是使用下面这个公式算出来的：

```
innodb_buffer_pool_size ÷ innodb_buffer_pool_instances
```

也就是 Buffer Pool 的总大小除以实例的个数，结果就是每个 Buffer Pool 实例占用的大小。

不过，并不是说 Buffer Pool 实例创建得越多越好，分别管理各个 Buffer Pool 也是需要性能开销的。设计 InnoDB 的大叔规定：当 innodb_buffer_pool_size 的值小于 1GB 时，设置多个实例是无效的，InnoDB 会默认把 innodb_buffer_pool_instances 的值修改为 1。

17.2.10　innodb_buffer_pool_chunk_size

在 MySQL 5.7.5 版本之前，只能在服务器启动时通过配置 innodb_buffer_pool_size 启动选项来调整 Buffer Pool 的大小；在服务器运行过程中是不允许调整该值的。不过设计 MySQL 的大叔在 MySQL 5.7.5 以及之后的版本中，支持了在服务器运行过程中调整 Buffer Pool 大小的功能。但是有一个问题，就是每次重新调整 Buffer Pool 的大小时，都需要重新向操作系统申请一块连续的内存空间，然后将旧 Buffer Pool 中的内容复制到这一块新空间；这是极其耗时的。所以，设计 MySQL 的大叔决定不再一次性为某个 Buffer Pool 实例向操作系统申请一大片连续的内存空间，而是以一个 chunk 为单位向操作系统申请空间。也就是说，一个 Buffer Pool 实例其实是由若干个 chunk 组成的。一个 chunk 就代表一片连续的内存空间，里面包含了若干缓冲页与其对应的控制块，如图 17-6 所示。

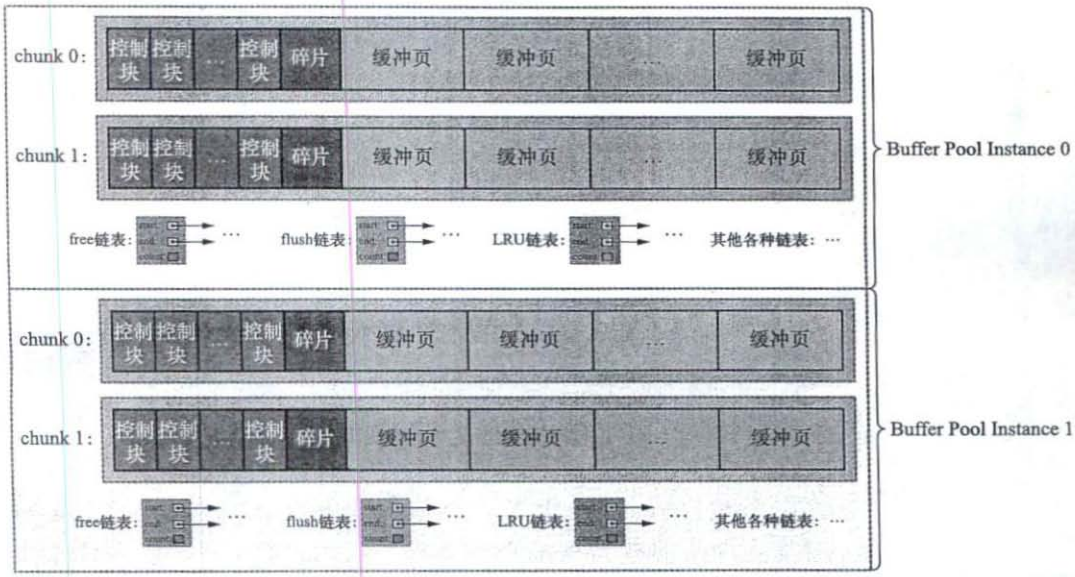

图 17-6 chunk 示意图

在图 17-6 中，Buffer Pool 就是由 2 个实例组成的，每个实例中又包含 2 个 chunk。

正是因为发明了 chunk 的概念，我们在服务器运行期间调整 Buffer Pool 的大小时，就可以以 chunk 为单位来增加或者删除内存空间，而不需要重新向操作系统申请一片大的内存，然后进行缓冲页的复制。这个 chunk 的大小是在启动 MySQL 服务器时，通过 innodb_buffer_pool_chunk_size 启动选项指定的，默认值是 134,217,728，也就是 128MB。不过需要注意的是，innodb_buffer_pool_chunk_size 的值只能在服务器启动时指定，在服务器运行过程中不可以修改。

小贴士

为什么不允许在服务器的运行过程中修改 innodb_buffer_pool_chunk_size 的值呢？还不是因为 innodb_buffer_pool_chunk_size 的值代表 InnoDB 向操作系统申请的一片连续的内存空间的大小。如果在服务器运行过程中修改了该值，就意味着需要重新向操作系统申请连续的内存空间，并且将原先的缓冲页和它们对应的控制块复制到这个新的内存空间中。这是十分耗时的操作！

另外，这个 innodb_buffer_pool_chunk_size 的值并不包含缓冲页对应的控制块的内存空间大小，所以实际上 InnoDB 向操作系统申请连续内存空间时，每个 chunk 的大小要比 innodb_buffer_pool_chunk_size 的值大一些（在 DEBUG 模式下约 5%）。

17.2.11 配置 Buffer Pool 时的注意事项

在配置 Buffer Pool 时，需要注意下面这些事项。

- innodb_buffer_pool_size 必须是 innodb_buffer_pool_chunk_size × innodb_buffer_pool_instances 的倍数（主要是想保证每一个 Buffer Pool 实例中包含的 chunk 数量相同）。

假设我们指定的 innodb_buffer_pool_chunk_size 的值是 128MB，innodb_buffer_pool_instances 的值是 16，那么这两个值的乘积就是 2GB，也就是说 innodb_buffer_pool_size 的值必须是 2GB 或者 2GB 的整数倍。比如，我们在启动 MySQL 服务器是按照下面这样来指定启动选项的：

```
mysqld --innodb-buffer-pool-size=8G --innodb-buffer-pool-instances=16
```

默认的innodb_buffer_pool_chunk_size值是128MB，指定的innodb_buffer_pool_instances的值是16，所以innodb_buffer_pool_size的值必须是2GB或者2GB的整数倍。在上面这个例子中，指定的innodb_buffer_pool_size的值是8GB，符合规定，所以在服务器启动完成之后，可以看到该变量的值就是我们指定的8GB（8,589,934,592字节）：

```
mysql> SHOW VARIABLES LIKE 'innodb_buffer_pool_size';
+-------------------------+------------+
| Variable_name           | Value      |
+-------------------------+------------+
| innodb_buffer_pool_size | 8589934592 |
+-------------------------+------------+
1 row in set (0.00 sec)
```

如果我们指定的innodb_buffer_pool_size大于2GB但不是2GB的整数倍，那么服务器会自动把innodb_buffer_pool_size的值调整为2GB的整数倍。比如我们在启动服务器时指定的innodb_buffer_pool_size的值是9GB：

```
mysqld --innodb-buffer-pool-size=9G --innodb-buffer-pool-instances=16
```

服务器会自动把innodb_buffer_pool_size的值调整为10GB（10,737,418,240字节），不信你看：

```
mysql> SHOW VARIABLES LIKE 'innodb_buffer_pool_size';
+-------------------------+-------------+
| Variable_name           | Value       |
+-------------------------+-------------+
| innodb_buffer_pool_size | 10737418240 |
+-------------------------+-------------+
1 row in set (0.01 sec)
```

- 在服务器启动时，如果innodb_buffer_pool_chunk_size × innodb_buffer_pool_instances的值已经大于innodb_buffer_pool_size的值，那么innodb_buffer_pool_chunk_size的值会被服务器自动设置为innodb_buffer_pool_size ÷ innodb_buffer_pool_instances的值。

比如，我们在启动服务器时指定的innodb_buffer_pool_size的值为2GB，innodb_buffer_pool_instances的值为16，innodb_buffer_pool_chunk_size的值为256MB：

```
mysqld --innodb-buffer-pool-size=2G --innodb-buffer-pool-instances=16 --innodb-buffer-pool-
chunk-size=256M
```

由于256MB × 16 = 4GB，而4GB>2GB，所以innodb_buffer_pool_chunk_size的值会被服务器改写为innodb_buffer_pool_size ÷ innodb_buffer_pool_instances的值，即2GB ÷ 16=128MB（134,217,728字节），不信你看：

```
mysql> SHOW VARIABLES LIKE 'innodb_buffer_pool_size';
+-------------------------+------------+
| Variable_name           | Value      |
+-------------------------+------------+
| innodb_buffer_pool_size | 2147483648 |
+-------------------------+------------+
```

```
1 row in set (0.01 sec)

mysql> SHOW VARIABLES LIKE 'innodb_buffer_pool_chunk_size';
+-------------------------------+-----------+
| Variable_name                 | Value     |
+-------------------------------+-----------+
| innodb_buffer_pool_chunk_size | 134217728 |
+-------------------------------+-----------+
1 row in set (0.00 sec)
```

Buffer Pool 中的缓冲页除了用来缓存磁盘中的页面以外，还可以存储自适应哈希索引的信息，这些内容就不再详细唠叨了。

17.2.12 查看 Buffer Pool 的状态信息

设计 MySQL 的大叔贴心地给我们提供了 SHOW ENGINE INNODB STATUS 语句来查看 InnoDB 存储引擎运行过程中的一些状态信息，其中就包括 Buffer Pool 的信息（为了突出重点，这里只把输出中与 Buffer Pool 相关的部分提取了出来）。

```
mysql> SHOW ENGINE INNODB STATUS\G

（……省略前边的许多状态）
----------------------
BUFFER POOL AND MEMORY
----------------------
Total memory allocated 13218349056;
Dictionary memory allocated 4014231
Buffer pool size   786432
Free buffers       8174
Database pages     710576
Old database pages 262143
Modified db pages  124941
Pending reads 0
Pending writes: LRU 0, flush list 0, single page 0
Pages made young 6195930012, not young 78247510485
108.18 youngs/s, 226.15 non-youngs/s
Pages read 2748866728, created 29217873, written 4845680877
160.77 reads/s, 3.80 creates/s, 190.16 writes/s
Buffer pool hit rate 956 / 1000, young-making rate 30 / 1000 not 605 / 1000
Pages read ahead 0.00/s, evicted without access 0.00/s, Random read ahead 0.00/s
LRU len: 710576, unzip_LRU len: 118
I/O sum[134264]:cur[144], unzip sum[16]:cur[0]
--------------
（……省略后边的许多状态）
```

我们来详细看一下里面的每个值都代表什么意思。

- Total memory allocated：代表 Buffer Pool 向操作系统申请的连续内存空间大小，包括全部控制块、缓冲页，以及碎片的大小。
- Dictionary memory allocated：为数据字典信息分配的内存空间大小。注意，这个内存空间和 Buffer Pool 没有关系，不包含在 Total memory allocated 中。
- Buffer pool size：代表该 Buffer Pool 可以容纳多少缓冲页。注意，单位是页！

- Free buffers：代表当前Buffer Pool还有多少空闲缓冲页，也就是free链表中还有多少个节点。
- Database pages：代表LRU链表中页的数量，它包含young和old两个区域的节点数量。
- Old database pages：代表LRU链表old区域的节点数量。
- Modified db pages：代表脏页数量，也就是flush链表中节点的数量。
- Pending reads：等待从磁盘加载到Buffer Pool中的页面数量。
  当准备从磁盘中加载某个页面时，会先在Buffer Pool中为这个页面分配一个缓冲页以及对应的控制块，然后把这个控制块添加到LRU的old区域的头部。但是此时真正的磁盘页并没有加载进来，因此Pending reads的值会加1。
- Pending writes LRU：即将从LRU链表中刷新到磁盘中的页面数量。
- Pending writes flush list：即将从flush链表中刷新到磁盘中的页面数量。
- Pending writes single page：即将以单个页面的形式刷新到磁盘中的页面数量。
- Pages made young：代表LRU链表中曾经从old区域移动到young区域头部的节点数量。
  这里需要注意，一个节点每次只有从old区域移动到young区域头部时才会将Pages made young的值加1。也就是说，如果该节点本来就在young区域，由于它符合在young区域1/4后面的要求，下一次访问这个页面时也会将它移动到young区域头部，但这个过程并不会导致Pages made young的值加1。
- Page made not young：在将innodb_old_blocks_time的值设置为大于0时，首次访问或者后续访问某个处于old区域的节点时，由于不符合时间间隔的限制而不能将其移动到young区域头部中，Page made not young的值会加1。
  这里需要注意，对于处于young区域的节点，如果因为它在young区域的前1/4处而没有被移动到young区域头部，Page made not young的值不会加1。
- youngs/s：代表每秒从old区域移动到young区域头部的节点数量。
- non-youngs/s：代表每秒由于不满足时间限制而不能从old区域移动到young区域头部的节点数量。
- Pages read、created、written：代表读取、创建、写入了多少页，后边跟着读取、创建、写入的速率。
- Buffer pool hit rate：表示在过去某段时间内，平均访问1000次页面时，该页面有多少次已经被缓存到Buffer Pool中。
- young-making rate：表示在过去某段时间内，平均访问1000次页面时，有多少次访问使页面移动到young区域的头部。
  需要注意的一点是，这里统计的将页面移动到young区域的头部次数不仅仅包含从old区域移动到young区域头部的次数，还包含从young区域移动到young区域头部的次数（访问某个young区域的节点时，只要该节点在young区域的1/4处后面，就会把它移动到young区域的头部）。
- not (young-making rate)：表示在过去某段时间内，平均访问1000次页面时，有多少次访问没有使页面移动到young区域的头部。
  需要注意的一点是，这里统计的没有将页面移动到young区域的头部次数不仅仅包含因设置了innodb_old_blocks_time系统变量而导致访问了old区域中的节点，但没把它们

移动到 young 区域的次数；还包含因为该节点在 young 区域的前 1/4 处而没有被移动到 young 区域头部的次数。

- LRU len：代表 LRU 链表中节点的数量。
- unzip_LRU：代表 unzip_LRU 链表中节点的数量（由于我们没有具体唠叨过这个链表，现在可以忽略它的值）。
- I/O sum：最近 50s 读取磁盘页的总数。
- I/O cur：现在正在读取的磁盘页数量。
- I/O unzip sum：最近 50s 解压的页面数量。
- I/O unzip cur：正在解压的页面数量。

17.3 总结

磁盘太慢，用内存作为缓冲区很有必要。

Buffer Pool 本质上是 InnoDB 向操作系统申请的一段连续的内存空间。可以通过 innodb_buffer_pool_size 来调整它的大小。

Buffer Pool 向操作系统申请的连续内存空间由控制块和缓冲页组成，每个控制块和缓冲页都是一一对应的。在填充了足够多的控制块和缓冲页的组合后，Buffer Pool 中剩余的空间可能不足以填充一组控制块和缓冲页，从而导致这部分空间无法使用。这部分空间也称为碎片。

InnoDB 使用了许多链表来管理 Buffer Pool。

在 free 链表中，每一个节点都代表一个空闲的缓冲页，在将磁盘中的页加载到 Buffer Pool 中时，会从 free 链表中寻找空闲的缓冲页。

为了快速定位某个页是否被加载到 Buffer Pool 中，可使用表空间号 + 页号作为 key，缓冲页控制块的地址作为 value 的形式来建立哈希表。

在 Buffer Pool 中，被修改的页称为脏页。脏页并不是立即刷新的，而是加入到 flush 链表中，待之后的某个时刻再刷新到磁盘中。

LRU 链表分为 young 区域和 old 区域，可以通过 innodb_old_blocks_pct 来调节 old 区域所占的比例。首次从磁盘加载到 Buffer Pool 中的页会放到 old 区域的头部，在 innodb_old_blocks_time 间隔时间内访问该页时，不会把它移动到 young 区域头部。在 Buffer Pool 中没有可用的空闲缓冲页时，会首先淘汰掉 old 区域中的一些页。

可以通过指定 innodb_buffer_pool_instances 来控制 Buffer Pool 实例的个数。每个 Buffer Pool 实例都有各自独立的链表，互不干扰。

自 MySQL 5.7.5 版本之后，可以在服务器运行过程中调整 Buffer Pool 的大小。每个 Buffer Pool 实例由若干个 chunk 组成，每个 chunk 的大小可以在服务器启动时通过启动选项调整。

可以用下面的命令来查看 Buffer Pool 的状态信息：

```
SHOW ENGINE INNODB STATUS\G
```